Streaming Communication for Scalable Data Exchange in DYNAMOS

Ruben Stoop
Complex Cyber-Infrastructure (CCI)
University of Amsterdam
Amsterdam, the Netherlands

***Abstract*—This thesis investigates how streaming communication can be introduced into DYNAMOS, a policy-driven middleware for data exchange based on dynamically composed microservices. The current system relies on unary gRPC request-response interactions, which become increasingly inefficient for large or continuous transfers. To ground the broader architectural and comparative study, the thesis first reviews candidate streaming mechanisms and evaluates them in a controlled *Producer* → *Transformer* → *Sink* pipeline. The current empirical results compare client-streaming gRPC, unary gRPC, RabbitMQ Streams, NATS JetStream, and Kafka in a single-repeat clean bulk-transfer matrix, with focused continuous, backpressure, recovery, and CSV-replay reruns now available for all five transports. Client-streaming provides the highest sustained throughput and unary keeps the lowest per-message latency. Among the brokered mappings, NATS JetStream becomes the strongest average performer only after explicit JetStream resource limits, async publish, batched pull consumption, and in-memory consumer-state tracking are configured. In the current clean rerun it averages 13,116.27 msg/s and 32.13 MiB/s, but it no longer remains duplicate-free: 5,086 duplicate deliveries appear in the 4 KiB/concurrency-16 and 16 KiB high-concurrency cells. RabbitMQ Streams remains a strong brokered reference, and Kafka moves into the middle of the pack once keyed partitioning, publisher flushing, and buffered consumer commits are configured. The final clean matrix still records a small number of duplicate deliveries in two 16 KiB Kafka cells, while the completed scenario reruns show that Kafka remains correctness-preserving under recovery but pays for that with heavier tail latency, slower backlog drain, and higher broker resource usage than the other brokered mappings. These findings define the current trade-off space for brokered versus direct streaming in DYNAMOS and motivate the remaining scenario-level and system-integration work.**

***Index Terms*—streaming communication, microservices, gRPC, event streaming, policy-driven middleware, DYNAMOS**

I. Introduction

Many contemporary distributed systems operate in environments where data and computational resources are owned by different organizations. In such settings, data exchange must adhere to legal agreements, privacy regulations, and organizational policies. These constraints are common in domains such as healthcare, scientific research, and public-sector infrastructures, where unrestricted data exchange is not acceptable.

DYNAMOS (Dynamically Adaptive Microservice-based Operating System) is a data exchange middleware developed by the Complex Cyber-Infrastructure (CCI) group at the University of Amsterdam. The system implements a set of recurring data sharing archetypes that describe how data and computation are distributed among participants under policy constraints. DYNAMOS realizes these archetypes by dynamically composing chains of microservices that execute data sharing requests in compliance with formally evaluated policies. The middleware is designed to be adaptive, allowing architectural and execution decisions to change at runtime in response to policy requirements.

The current implementation of DYNAMOS uses gRPC for inter-service communication and relies exclusively on atomic request–response interactions. This communication model is suitable for small or bounded data transfers but becomes limiting when applied to large-scale or continuous data exchange. In practice, data in distributed systems is often produced and consumed incrementally, for example in sensor data streams, distributed analytics pipelines, or staged processing of large research datasets.

When request–response communication is used in such scenarios, systems must either buffer large volumes of data before transmission or split transfers into multiple independent requests. Buffering increases memory usage and can lead to idle time between producers and consumers. Repeated request–response interactions introduce additional coordination overhead. Both effects negatively impact scalability and resource utilization as data volumes and the number of participating services increase.

Streaming-based communication has been proposed as a solution to these limitations. Streaming allows data to be transferred and processed incrementally, enabling overlap between computation and communication and reducing peak memory usage. gRPC supports several streaming interaction patterns, including client-side, server-side, and bidirectional streaming.

Integrating streaming communication into DYNAMOS is not straightforward. Streaming relies on long-lived connections and continuous data flows, which must be reconciled with DYNAMOS' policy-driven execution model, dynamic microservice composition, orchestration decisions, and fault handling mechanisms. Without careful design, the introduction of streaming may interfere with policy enforcement or reduce the ability to evaluate and reason about system behavior.

The goal of this thesis is to investigate how streaming communication can be incorporated into DYNAMOS in a way that improves scalability and performance while preserving

policy compliance and adaptive behavior. This involves analyzing existing streaming communication mechanisms, designing architectural extensions to the middleware, implementing streaming support, and evaluating the resulting system under realistic workloads.

The main research question addressed in this thesis is how streaming communication can be integrated into a dynamically adaptive, policy-driven microservice-based data exchange middleware such as DYNAMOS while preserving compliance guarantees and improving scalability and performance. To answer this overarching question, the thesis is structured around three sub-questions: which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS; how streaming communication can be integrated into a microservice-based architecture while preserving dynamic service composition, policy-driven orchestration, and isolation guarantees; and what impact streaming communication has on scalability, performance, and robustness compared to a traditional request–response approach under realistic workloads in a microservice-based architecture.

The present document concentrates on the first empirical step in that broader programme. It narrows the design space through a literature-guided comparison, defines a reproducible benchmark for DYNAMOS-relevant streaming workloads, and reports the current five-transport clean-matrix benchmark results for client-streaming gRPC, unary gRPC, RabbitMQ Streams, NATS JetStream, and Kafka. Later thesis stages build on this comparison by extending scenario coverage and by evaluating architectural fit inside DYNAMOS itself.

II. Related Work

This section reviews existing literature related to streaming communication in microservice-based systems, with a particular focus on topics that are central to this thesis: streaming communication technologies and interaction models, architectural integration in adaptive microservice middleware, policy enforcement under continuous data exchange, resilience mechanisms for streaming workflows, and validation practices with respect to scalability and performance.

A. Streaming communication technologies and interaction models

DYNAMOS is implemented as a dynamically composed microservice system deployed on Kubernetes, using gRPC for inter-service communication and RabbitMQ for asynchronous messaging [1], [2]. This constrains the design space for introducing streaming functionality to a limited set of technologies that are compatible with containerized microservices and cloud-native deployment models. The primary question addressed in this body of work is which streaming communication mechanisms are suitable for large-scale or continuous data exchange, and under which conditions direct streaming RPC should be preferred over broker-based approaches.

gRPC provides native support for streaming through client-streaming, server-streaming, and bidirectional streaming RPCs. These interaction patterns differ in message flow control, lifetime of connections, buffering behaviour, and error propagation semantics. In the context of DYNAMOS, the choice of streaming pattern is not merely an API concern but has architectural consequences, as it affects memory usage, cancellation behaviour, and the ability to overlap communication and computation. The original DYNAMOS thesis identifies client-streaming as a promising mechanism for transferring large datasets incrementally, reducing peak memory pressure at the cost of increased handling complexity [1].

However, the use of long-lived streaming connections introduces operational challenges that are largely absent in unary request–response interactions. Starck and Ghofrani show that gRPC streaming complicates load balancing in Kubernetes environments, as persistent connections can cause traffic to become unevenly distributed across replicas [3]. This observation is particularly relevant for systems such as DYNAMOS that rely on horizontal scaling and ephemeral execution units. Complementary benchmarking work demonstrates that the performance characteristics of streaming gRPC are sensitive to runtime configuration and virtualization overhead, with measurable effects on latency and resource utilisation in containerized environments [4].

An alternative to direct service-to-service streaming is to externalise the data plane to a messaging or event streaming platform. Broker-based communication decouples producers and consumers in time, delegates buffering and persistence to the broker, and can simplify fan-out scenarios. Comparative benchmarks of Kafka, RabbitMQ, NATS, and ActiveMQ Artemis show that different brokers dominate under different workload characteristics, reinforcing that broker choice is highly workload dependent [5]. A cloud-native literature review comparing Kafka, Pulsar, and RabbitMQ further highlights trade-offs between throughput, reliability, operational complexity, and Kubernetes integration [6]. While these studies are not conducted in the context of policy-driven middleware, they provide structured criteria that are directly applicable when evaluating broker-based alternatives for DYNAMOS.

Several studies also emphasize that streaming design is influenced not only by technology choice but by the underlying communication model. DataXc contrasts push-based streaming approaches with pull-based designs and reports improved latency stability and throughput under consumer-controlled flow [7]. Although the application domain differs, the distinction between producer-driven and consumer-driven streaming is directly relevant to backpressure management and resource utilisation in DYNAMOS-compatible workflows.

Overall, existing literature provides a broad comparison of streaming technologies and interaction patterns, but these comparisons are typically conducted in generic microservice or stream-processing settings. There is limited work that evaluates these mechanisms under the specific constraints of dynamically composed, policy-driven middleware with eph-

emeral execution units, leaving open questions regarding their suitability in such environments.

B. Architectural integration of streaming in adaptive microservice middleware

Beyond the selection of a streaming mechanism, integrating streaming communication into an adaptive microservice architecture raises architectural questions related to modularity, orchestration, and lifecycle management. DYNAMOS is explicitly designed around dynamic microservice composition, where execution chains are generated and deployed per request based on policy evaluation and system state [1], [2]. These chains are short-lived by design, which improves isolation and compliance guarantees but complicates the management of long-lived communication channels.

Communication in DYNAMOS is abstracted through a sidecar-based architecture, decoupling communication concerns from core microservice logic. The sidecar pattern is widely used in cloud-native systems to address cross-cutting concerns such as observability, security, and traffic management. Figueira and Coutinho demonstrate how sidecars can be integrated into self-adaptive microservice architectures on Kubernetes, enabling runtime adaptation without violating service modularity [8]. While their work does not explicitly address streaming data transfer, it illustrates how communication behaviour can be adapted dynamically at runtime.

Several middleware frameworks operationalise sidecars as first-class runtime components. Dapr provides a protocol-agnostic communication abstraction that supports service invocation, pub/sub, and state management through a sidecar runtime [8]. Dapr demonstrates that heterogeneous communication paradigms, including streaming and messaging, can be unified behind a stable interface. However, Dapr assumes relatively static service topologies and does not address dynamic chain composition driven by policy evaluation, limiting its applicability as a direct solution for DYNAMOS.

Existing architectural work therefore supports the feasibility of integrating streaming into sidecar-based microservice systems but does not address the combination of long-lived streaming connections with ephemeral, dynamically composed execution chains. This limitation is particularly relevant in systems that combine adaptivity with strict policy enforcement requirements.

C. Policy enforcement and compliance under continuous data exchange

Policy enforcement is a defining characteristic of DYNAMOS, where access rights, execution locations, and data-flow constraints are evaluated prior to execution and enforced through dynamically composed microservice chains [1], [2]. In the current architecture, policy enforcement is implicitly scoped to bounded request–response interactions. Introducing streaming communication challenges this assumption, as data transfer may span longer periods during which policies, system state, or execution context may change.

Neumaier et al. propose a policy-aware architecture for decentralized dataset exchange that emphasizes continuous monitoring and runtime validation against machine-readable policies [9]. While their work does not explicitly target streaming RPC, it introduces an important distinction between one-time policy clearance and ongoing compliance monitoring, which becomes critical when data is exchanged incrementally over long-lived connections.

At the communication layer, several studies propose enforcing security and access control through proxies or sidecars. Thiyagarajan et al. present a proxy-based approach for securing gRPC communication using mTLS, JWT authentication, and RBAC [10]. Although the focus is on security rather than data usage policy, the architectural approach is relevant, as it suggests that enforcement can be applied at communication boundaries and potentially at message granularity.

Despite these contributions, there is limited guidance on how policy enforcement should interact with long-lived streaming connections in systems that dynamically compose execution chains. In particular, existing work does not address how policy violations should be detected or handled mid-stream, or how streams should be terminated or adapted when compliance conditions change. These limitations highlight open challenges when introducing streaming into policy-driven middleware such as DYNAMOS.

D. Resilience, backpressure, and fault handling in streaming workflows

Streaming-based communication introduces operational challenges that are less pronounced in request–response systems. Long-lived data flows must tolerate partial failures, adapt to fluctuating load, and prevent uncontrolled resource growth through effective backpressure mechanisms. These challenges are compounded in DYNAMOS by dynamic microservice composition and ephemeral execution units.

Reactive microservice architectures emphasize asynchronous communication, non-blocking execution, and fault isolation as mechanisms for improving resilience under variable load [11]. Although this work does not focus on streaming RPC, its principles are directly applicable to streaming workflows, where failures and backpressure must be managed continuously rather than per request.

Studies on high-volume data processing in microservices highlight the importance of explicit flow control and decoupling between producers and consumers. Guntupalli and Vamshi show that event-driven architectures combined with messaging systems can mitigate bottlenecks and stabilize performance under sustained data rates [12]. While these systems typically rely on brokers rather than direct streaming RPC, they reinforce the need for explicit backpressure mechanisms in streaming data paths.

From a systems perspective, Gan and Delimitrou demonstrate that communication overhead and tail latency dominate the performance of large-scale microservice deployments [13]. Their findings underline that resilience and scalability issues

often emerge from inter-service communication rather than computation, an effect that is amplified in streaming scenarios.

Existing literature provides well-established principles for resilience and fault tolerance in microservice systems, but these are typically studied either in request-oriented architectures or broker-based streaming platforms. There is limited work examining how these mechanisms interact with direct streaming RPC in dynamically composed microservice chains, leaving questions about how failures and backpressure should be handled in dynamically composed microservice systems.

E. Validation practices, scalability, and performance evaluation

Empirical evaluation of streaming communication often focuses on comparing streaming and non-streaming interaction styles in terms of throughput, latency, and resource usage. Several benchmarking studies show that streaming RPCs outperform unary calls for large payloads and continuous data flows by enabling incremental processing and reducing buffering overhead [4]. Similar conclusions are drawn in broader comparisons between REST and gRPC, where binary protocols combined with streaming semantics demonstrate superior performance under load [14], [15].

Scalability in containerized streaming systems has been studied through systematic mapping and empirical evaluations. A mapping study on performance analysis techniques for streaming workloads in containerized microservices identifies network contention, scheduling overhead, and coordination costs as primary scalability bottlenecks [16]. Empirical studies of streaming pipelines deployed on Kubernetes further show that streaming approaches scale more gracefully than request-response models when concurrency increases, provided that backpressure and flow control are properly configured [3].

However, most existing evaluations assume static microservice topologies and fixed execution paths. Prior performance studies rarely consider adaptive middleware that dynamically constructs execution chains based on policy evaluation. As a result, there is limited empirical insight into how streaming communication scales when embedded within dynamically adaptive, policy-driven systems such as DYNAMOS. This lack of combined evaluation of streaming, scalability, and adaptivity motivates the experimental focus of this thesis.

III. Candidate Mechanisms and Benchmark Design

This section addresses *RQ1*: which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS? It combines a literature-backed comparison of candidate mechanisms with a controlled benchmark design that establishes a gRPC reference baseline before broker-based alternatives are added.

A. gRPC streaming

According to the official gRPC documentation, gRPC is based on service definitions in Protocol Buffers and supports four RPC styles: unary, server-streaming, client-streaming, and bidirectional streaming. For streaming calls, message ordering is preserved within each individual RPC, while deadlines, cancellation, metadata, and both synchronous and asynchronous APIs are supported by the runtime [17]. These properties make gRPC streaming the most direct extension of the current DYNAMOS communication model.

For microservice chains with predominantly point-to-point data transfer, client-streaming is the most natural first candidate because it lets a sender incrementally push chunks or records to a single downstream consumer. Bidirectional streaming becomes relevant when the receiver should provide progress feedback, pacing information, or explicit acknowledgements during transfer. The main advantages of gRPC streaming are low protocol overhead, direct reuse of protobuf contracts, and the absence of an additional broker layer. The disadvantages are that sender and receiver remain temporally coupled, durable replay is not provided by default, and backlog handling or crash recovery must be implemented at application level. Even so, prior work on asynchronous gRPC streaming demonstrates that carefully designed protocols can support efficient and even exactly-once style delivery patterns, which makes gRPC a strong experimental baseline rather than merely the status-quo option [4], [14], [18].

B. Apache Kafka

Apache Kafka is officially described as an open-source distributed event-streaming platform for high-performance pipelines, streaming analytics, data integration, and mission-critical applications. Its published core capabilities emphasise high throughput, durable storage, high availability, built-in stream processing, and large-scale horizontal scalability [19]. In architectural terms, Kafka exposes an append-only log abstraction in which messages are written to topics and partitions, allowing replay and independent consumption by multiple downstream readers [20].

Kafka is therefore the strongest candidate when temporal decoupling, replayability, or large consumer backlogs matter more than direct point-to-point simplicity. Its advantages are durable retention, repeated reads by independent consumers, and a mature ecosystem for analytics and downstream integration. The disadvantages are the extra operational footprint of a broker cluster, partition-local rather than global ordering, and a non-trivial tuning burden around acknowledgements, partitioning, batching, and replication. Recent review work also notes that Kafka benchmarks are frequently hard to reproduce because configuration details are underreported, which strengthens the case for a carefully controlled thesis benchmark [20]–[22].

C. RabbitMQ

Earlier comparative work positions RabbitMQ as a routing-oriented messaging system that offers flexible exchanges, acknowledgements, and strong support for enterprise-style asynchronous messaging [20], [23]. Current RabbitMQ documentation additionally introduces *streams* as an append-only,

non-destructive, always persistent and replicated data structure that complements traditional queues rather than replacing them. Stream consumers can attach at arbitrary offsets or timestamps, replay data, and scale out via super streams when partitioning is needed [24].

This makes RabbitMQ attractive as a hybrid comparison point between classic broker semantics and newer log-like stream semantics within the same ecosystem. Its advantages are flexible routing, explicit acknowledgements, and a gradual migration path from queue-oriented messaging to append-only streams. The limitations are that classic RabbitMQ semantics are not optimised for replay-heavy logs, while stream mode introduces its own replication and partitioning trade-offs. RabbitMQ should therefore be evaluated not only on raw throughput, but also on how well it balances routing flexibility, operational continuity, and stream-specific behaviour [20], [23], [24].

D. NATS with JetStream persistence

The NATS ecosystem deserves separate consideration because it targets lightweight, location-transparent distributed communication. Official NATS documentation presents core NATS as a connective technology for modern distributed systems with publish/subscribe and request/reply communication, while JetStream adds persistence, replay, replication, and consumer state on top of the base message bus [25]–[27]. JetStream explicitly supports replay from the beginning of a stream, from a specific sequence number, or from a time-based position, and it offers both push-based and pull-based consumers for different workload profiles [27].

For the thesis, NATS is interesting because it represents a lightweight alternative to Kafka and RabbitMQ rather than a direct substitute for either. JetStream supports durable streams, replay, replication, and scalable pull consumers, while core NATS preserves a simple request/reply and pub/sub model that is attractive for microservice systems [26], [27]. A key complication is that the literature still often discusses the older NATS Streaming layer rather than JetStream, which means published results do not always map perfectly to the current product. That mismatch should be made explicit in the thesis: the experiment should evaluate modern JetStream, but the literature review must interpret older NATS Streaming results with caution [22], [23]. NATS with JetStream is most promising when lightweight deployment, elastic scaling, and a smaller operational surface are valued more highly than Kafka's larger ecosystem and benchmarking history.

E. Benchmark Design

1) Methodological positioning: The benchmark design is not presented as a standard protocol copied directly from one prior study, nor as a completely novel framework. Instead, it adapts recurring benchmarking principles from streaming-system evaluation, distributed stream processing studies, and comparative IPC experiments to the specific constraints of DYNAMOS. In particular, the controlled multi-stage pipeline follows the style of prior streaming-pipeline evaluations such as DataXc [7]; the insistence on identical business logic across mechanisms follows comparative benchmarking practice in distributed systems [28], [29]; and the emphasis on throughput, latency, concurrency, resource pressure, and recovery reflects recurring evaluation dimensions in the literature [16], [22]. The resulting design is therefore best understood as a literature-guided, thesis-specific operationalization of established evaluation concerns.

2) Evaluation scope: The purpose of this benchmark is not to identify a universally best communication mechanism in the abstract. Instead, it narrows the design space for DYNAMOS by asking which communication model best fits which workload condition and which operational constraint. The central question is therefore: *which streaming mechanism best supports large, ordered, and concurrent inter-service data transfer in a microservice pipeline under varying workload intensity, resource pressure, and recovery requirements?*

This framing matters for interpretation. The chapter body focuses on the argument and on the most informative comparisons, while exhaustive preset definitions, full run matrices, and supplementary plots are better treated as appendix material once the multi-technology comparison is complete.

3) Common testbed and fairness constraints: To ensure that observed differences can be attributed to the communication mechanism rather than to application logic, all candidate mechanisms should be evaluated with the same payload schema, the same transformation logic, and the same deployment environment. The common testbed is a Go-based microservice chain deployed on Kubernetes with three services: *Producer* $\rightarrow$ *Transformer* $\rightarrow$ *Sink*. The services are deployed in a dedicated namespace using Kustomize, with base manifests defining the common topology and overlay patches controlling the resource budget per experiment tier (CPU and memory requests and limits). Resource consumption is sampled at regular intervals via the Kubernetes metrics-server using `kubectl top`. This topology is small enough to reason about directly, but still rich enough to expose backpressure, buffering, and multi-hop propagation effects [7], [28].

The mechanism-specific mappings should remain as close as possible to one another. For gRPC, the default implementation should use client-streaming, with bidirectional streaming reserved for an extended scenario. For Kafka, messages should be keyed by flow identifier so that per-flow ordering is preserved within a partition. For RabbitMQ, the preferred setup is a stream rather than a classic queue, with super streams used only if scale-out beyond a single stream is necessary. For NATS, JetStream should be used with explicit acknowledgements and pull consumers. Holding business logic and payload shape constant is essential, because otherwise performance differences could be caused by application behaviour instead of the communication layer itself [28], [29].

The service logic is intentionally simple. The producer emits synthetic records or chunked data, the transformer applies a deterministic lightweight operation such as validation, filtering, or hashing, and the sink records arrival times, latency sum-

maries, and correctness counters. This keeps the benchmark focused on communication behaviour rather than on domain-specific business processing.

4) Evaluation dimensions: The literature supports evaluating streaming mechanisms along a small set of recurring dimensions rather than through a single headline metric. In this benchmark, five dimensions are treated as primary: transfer efficiency, meaning how much data the pipeline can move per second under comparable conditions; latency and pacing stability, meaning not only how fast messages arrive but how regularly they arrive under sustained flow [7]; overload behaviour, meaning whether pressure is absorbed through queue growth, throttling, or admission failure; recovery and correctness, meaning how reconnects, retries, duplicates, and ordering behave when a running pipeline is interrupted; and resource and operational cost, meaning CPU, memory, network traffic, and the practical complexity of running the mechanism in Kubernetes.

This dimensional view is important because it prevents the chapter from collapsing into a single throughput comparison. A mechanism may lead on bulk transport while still performing poorly under restart or overload, and such trade-offs are exactly what DYNAMOS must reason about.

5) Scenario set and scenario-to-property mapping: The full benchmark design distinguishes three workload families: *bulk transfer*, *continuous steady-rate flow*, and *bursty transfer*. The currently completed gRPC reference study covers the first two directly and supplements them with two targeted stress probes that operationalize the properties most relevant to DYNAMOS: slow-consumer backpressure and forced-restart recovery. A bursty profile remains part of the planned cross-technology matrix, but it has not yet been executed in the current gRPC-only reference stage.

6) Experimental factors: The independent variables cover the stress dimensions most frequently discussed in streaming-system and IPC benchmarking literature [16], [22], [29]: dataset size, to expose when buffering or backlog effects become significant; chunk or message size, to test the trade-off between per-message overhead and coarse-grained transfer latency; number of concurrent flows, to assess how each mechanism handles simultaneous transfers; pipeline depth, fixed here at a three-service chain so that raw transport overhead and multi-hop effects can still be distinguished; consumer speed mismatch, to expose backlog behaviour and flow-control effectiveness; resource budget, through explicit CPU and memory requests and limits for Kubernetes pods, defined as Kustomize overlay patches at three tiers (small: 100 m CPU / 128 MiB, medium: 250 m CPU / 256 MiB, large: 500 m CPU / 512 MiB); durability mode and replication factor, where the mechanism supports such trade-offs; and fault scenario, at minimum service restart during an ongoing transfer.

Not every factor needs to be crossed exhaustively. A pragmatic design is to define a core matrix over mechanism, workload profile, resource budget, and concurrency, and then use targeted follow-up experiments for failure and durability scenarios. This keeps the study manageable while still covering the trade-offs the literature treats as most important [22], [23].

7) Result presentation and appendix boundary: The body of the thesis should present only the compact experiment matrix and a smaller set of focused figures that are necessary for the argument. A master table can summarize each run by mechanism, workload profile, dataset size, message size, concurrency level, resource budget, and durability mode, together with the most important metrics. The main text should then highlight the most informative sweeps, such as throughput versus concurrency, latency percentiles versus message size, and recovery behaviour by failure scenario. Full preset definitions, per-run outputs, and secondary plots should be moved to an appendix so that the chapter remains analytically clear rather than turning into an undifferentiated lab log. In the current draft, Appendix B now carries the detailed per-case clean-matrix tables, while the body keeps the argument focused on cross-transport averages and the most decision-relevant slices.

F. gRPC Baseline Results

1) Metrics definitions: Before presenting results, it is important to define the metrics used throughout the evaluation so that tables, figures, and prose refer to consistent quantities.

- **Throughput (msg/s)** is the number of messages successfully received by the sink per second of wall-clock time, averaged over the entire run.
- **Throughput (MiB/s)** is the data volume received by the sink per second, computed as $(\mathrm{msg/s}) \times \mathrm{message\ size}$.
- **Latency percentiles (p_{50}, p_{95}, p_{99}, max)** describe end-to-end message delivery delay from producer send to sink receipt. The p_{95} latency, for example, means that 95% of messages were delivered within that time or less.
- **Inter-arrival time** is the elapsed time between consecutive message arrivals at the sink. The p_{95} inter-arrival time captures near-worst-case spacing between deliveries.
- **Jitter** is defined as the standard deviation of the inter-arrival times. Lower jitter indicates a more regular, predictable delivery rhythm.
- **Duplicates** is the number of messages received more than once by the sink. In a correct delivery, this count is zero.
- **Ordering violations** is the number of messages received out of their original send order.
- **CPU usage (%)** is the average and peak CPU consumption per role (producer, transformer, sink), sampled at two-second intervals via `kubectl top pods` during the focused Kubernetes reruns and computed as a percentage of a single core.
- **Memory usage (MiB)** is the average and peak resident memory per role, sampled on the same interval.
- **Time to first post-failure message (ms)** is the wall-clock time between the estimated failure instant and the first message that reaches the sink afterwards.
- **End-to-end recovery time (ms)** is the time required for the sink to observe three consecutive one-second windows at or above 90% of the target rate. If that

Table I
SCENARIO-TO-PROPERTY MAPPING USED IN THE BENCHMARK DESIGN.

Scenario	Primary property	Why it matters for DYNAMOS
Bulk transfer baseline	Protocol overhead, sustained throughput, multi-hop transfer efficiency	Large dataset exchange should remain efficient when data is moved incrementally through a composed service chain.
Continuous steady-rate flow	Latency stability, inter-arrival regularity, pacing precision, CPU cost	Long-lived flows matter when data is produced incrementally and consumers depend on relatively stable delivery rhythm.
Slow-consumer backpressure	Queue build-up, overload propagation, throttling versus delay accumulation	DYNAMOS pipelines may contain stages with unequal service rates, so overload handling must be explicit and interpretable.
Forced-restart recovery	Reconnect behaviour, duplicates, ordering preservation, recovery time	Ephemeral microservices and restarts are normal in Kubernetes-style deployments, so transport behaviour under interruption is a first-class concern.

threshold is never met before the run completes, the metric is capped at the backlog-drain time and reported together with a boolean indicating that the target rate was not recovered.

- **Backlog drain time (ms)** is the time between the estimated failure instant and the arrival of the final message for the run.

2) Current comparison scope: The benchmark harness now covers five transport mappings in the same three-stage pipeline (*Producer* → *Transformer* → *Sink*): *client-streaming gRPC*, *unary gRPC*, *RabbitMQ Streams*, *NATS JetStream*, and *Kafka*. The two gRPC variants remain informative because unary approximates the current direct request-response style used in DYNAMOS, whereas client-streaming represents the most direct streaming extension of that model. RabbitMQ Streams, NATS JetStream, and Kafka add broker-mediated reference points in which the producer publishes into a durable stream or topic and downstream stages consume through the broker rather than over a single long-lived RPC.

The current empirical corpus is a completed single-repeat `synthetic-clean` matrix executed on the same single-node Kubernetes environment for all five transports. It uses synthetic payloads with message sizes from 256 bytes to 16 KiB and concurrency levels 1, 4, 8, and 16, producing 80 completed runs in total. The sink records throughput, end-to-end latency percentiles, inter-arrival behaviour, and correctness counters for every run. Resource utilisation is sampled through `kubectl top` and exported with the run summaries.

This execution scope makes the current tables directly comparable across all five implemented transports, but it is still a single-pass benchmark suite rather than a repeated statistical study. The values reported below therefore establish a coherent

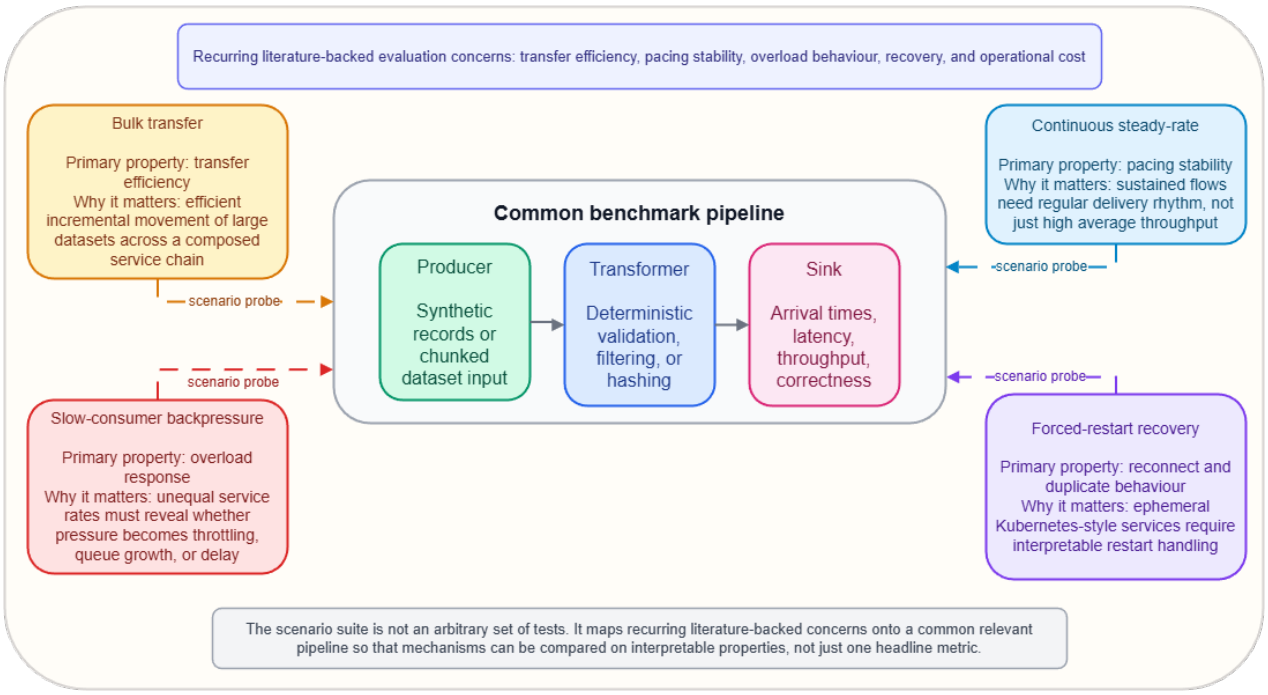

Figure 1. Evaluation map showing the common benchmark pipeline and the four scenario probes used to operationalize the literature-backed evaluation dimensions.

Table II
COMPLETED SYNTHETIC-CLEAN SUMMARY FOR THE SINGLE-REPEAT FIVE-TRANSPORT BENCHMARK SUITE.

Transport mode	Avg. msg/s	Avg. MiB/s	Avg. p_{95}	Avg. p_{99}	Duplicates	Ordering viol.
Client-streaming	31,259.76	51.35	242.82	441.59	0	0
Unary	2,291.37	9.55	3.01	31.23	0	0
RabbitMQ Streams	9,110.73	16.29	2,123.56	2,303.05	0	0
NATS JetStream	13,116.27	32.13	1,581.11	1,659.99	5,086	0
Kafka	7,354.42	20.12	1,729.34	1,901.42	9	0

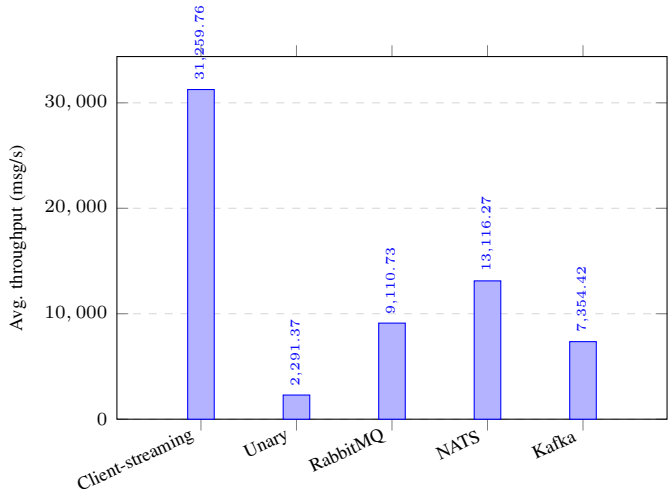

Figure 2. Average throughput for the completed single-repeat synthetic-clean matrix. Client-streaming sustains the highest average throughput overall, while NATS JetStream is the fastest brokered transport on average in the current clean rerun.

comparison point for the present prototype, but they should be interpreted as point estimates rather than variance-aware performance envelopes. Focused continuous, backpressure, recovery, and CSV-replay scenarios have now been rerun across all five transports, so the scenario-level comparison is complete as a first single-pass benchmark corpus even though repeated runs are still needed before variance-aware claims are made.

3) Clean baseline matrix: Table III breaks the synthetic bulk-transfer results down by message size, averaging across concurrency levels 1, 4, 8, and 16 for each size. This disaggregation shows that the client-streaming advantage is strongly size-dependent: it delivers roughly 29.2× the throughput of unary at 256 B, but that advantage narrows to about 3.8× at 16 KiB as the unary path amortises part of its per-call overhead. The brokered transports now separate more clearly. NATS JetStream gives the highest brokered average throughput at every message size in the refreshed clean rerun, while RabbitMQ Streams remains a strong reference at 256 B and 1 KiB and Kafka remains competitive without retaking the brokered lead. The earlier pathological 1 KiB/concurrency-16 NATS cliff no longer reproduces; the main NATS caveat in the current rerun is delivery semantics rather than completion, with duplicate deliveries now emerging in the heavier rows.

Table IV reports the same rows as data throughput. This view is important because DYNAMOS data exchange is often limited by transferred volume rather than only by message count.

Table III
SYNTHETIC BULK-TRANSFER MESSAGE THROUGHPUT BY MESSAGE SIZE, AVERAGED ACROSS CONCURRENCY LEVELS 1, 4, 8, AND 16 IN THE COMPLETED SINGLE-REPEAT SUITE.

Msg size	Stream msg/s	Unary msg/s	RabbitMQ msg/s	NATS msg/s	Kafka msg/s
256 B	75,604	2,586	13,065	22,561	12,467
1 KiB	27,682	2,599	14,381	15,432	7,573
4 KiB	15,358	2,319	7,905	10,086	6,527
16 KiB	6,395	1,662	1,092	4,387	2,850

Table IV
SYNTHETIC BULK-TRANSFER DATA THROUGHPUT BY MESSAGE SIZE, AVERAGED ACROSS CONCURRENCY LEVELS 1, 4, 8, AND 16 IN THE COMPLETED SINGLE-REPEAT SUITE.

Msg size	Stream MiB/s	Unary MiB/s	RabbitMQ MiB/s	NATS MiB/s	Kafka MiB/s
256 B	18.46	0.63	3.19	5.51	3.04
1 KiB	27.03	2.54	14.04	15.07	7.40
4 KiB	59.99	9.06	30.88	39.40	25.50
16 KiB	99.92	25.97	17.06	68.55	44.53

Three observations stand out. First, client-streaming throughput in msg/s drops substantially with payload size (from 75,604 to 6,395), but its data rate still rises from 18.46 to 99.92 MiB/s because larger messages amortise per-message overhead more effectively. Unary msg/s is much flatter across sizes (1,662–2,599), indicating that per-call overhead remains its main bottleneck. Second, RabbitMQ Streams performs much better in the final Kubernetes clean run than in the earlier local shakedown results, averaging 9,110.73 msg/s overall and 16.29 MiB/s. It still pays for brokered decoupling with much higher latency than unary, but it is no longer merely a low-throughput reference point. Third, NATS JetStream and Kafka expose different configuration sensitivities. NATS completed the full matrix only after JetStream memory and file-store limits were made explicit and aligned with the Kubernetes resource envelope. With async publish, batched pull consumption, and in-memory JetStream consumer state while keeping the stream itself file-backed, the refreshed NATS clean rerun now averages 13,116.27 msg/s and 32.13 MiB/s overall, overtaking RabbitMQ Streams and Kafka on average throughput in this single-pass suite. The main NATS caveat has shifted from a single pathological queueing cell to delivery semantics under heavier loads: the current rerun records 5,086 duplicate deliveries in total, concentrated in the 4 KiB concurrency-16 row and the 16 KiB concurrency-4, concurrency-8, and concurrency-16 rows, even though ordering violations remain zero across the clean matrix. Kafka no longer collapses in throughput after explicit key-hash partitioning, periodic publisher flushing, and buffered consumer commits: it now aver-

Table V
PEAK BROKER RESOURCE USE OBSERVED IN THE SINGLE-REPEAT SYNTHETIC-CLEAN MATRIX. DIRECT GRPC MODES HAVE NO EXTERNAL BROKER ROLE.

Brokered mode	Peak broker CPU (%)	Peak broker memory (MiB)
RabbitMQ Streams	465.10	510
NATS JetStream	11.20	15
Kafka	69.40	395

ages 7,354.42 msg/s and 20.12 MiB/s overall, overtakes unary in average throughput, and remains architecturally relevant as the durable-log reference. The remaining Kafka caveat is still tail behaviour and semantics rather than ordering: ordering violations dropped to zero across the final matrix, but the 16 KiB concurrency-4 and concurrency-16 cells still recorded nine duplicate deliveries in total.

All completed clean-matrix runs reached the target message count, and ordering violations were eliminated for every transport in the final `synthetic-clean` preset. The remaining correctness anomalies are now concentrated in the brokered at-least-once mappings: Kafka still records nine duplicate deliveries, concentrated in the 16 KiB concurrency-4 and concurrency-16 cells, while NATS JetStream records 5,086 duplicate deliveries concentrated in the 4 KiB concurrency-16 and 16 KiB high-concurrency rows. The current baseline trade-off therefore includes a small duplicate caveat for Kafka and a materially larger duplicate caveat for NATS in addition to throughput, latency, buffering, broker configuration effort, and resource cost. The full per-case clean-matrix values are listed in Tables XI–XIV in Appendix B.

The clean matrices alone, however, are not enough to judge whether a transport is suitable for long-lived, policy-constrained data exchange. DYNAMOS-style workflows also care about pacing regularity, behaviour under consumer mismatch, and failure recovery. Three focused stress scenarios were therefore added after the baseline.

4) Diagnostic focused stress scenarios: The following focused stress results now combine the earlier diagnostic suite for client-streaming gRPC, unary gRPC, and RabbitMQ Streams with refreshed reruns for NATS JetStream and Kafka. They remain useful because they explain why the benchmark records pacing, backpressure, recovery, and duplicate metrics rather than only clean throughput. The tables below therefore form the first completed single-repeat five-transport scenario comparison for the current prototype.

a) Continuous steady-rate scenario.: The first focused experiment was a continuous steady-rate scenario. It used 1 KiB synthetic messages, a fixed target rate of 1,000 messages per second, and concurrency levels of 4 and 8, without artificial sink delay or injected failures. The goal was to measure not just throughput and latency, but also inter-arrival regularity under a long-running steady stream.

In this single-repeat continuous scenario, both gRPC modes stay near the 1,000 msg/s target, but they shape the flow differently. Client-streaming maintains the tighter delivery rhythm in both concurrency settings: its jitter remains near 0.66–0.67 ms

Table VI
CONTINUOUS STEADY-RATE SCENARIO FROM THE COMPLETED SINGLE-REPEAT FIVE-TRANSPORT SUITE.

Transport mode	Concur.	Throughput	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	4	1,016.80	8.82	1.59	0.66
Client-streaming	8	995.35	6.85	1.55	0.67
Unary	4	1,049.43	1.66	2.12	2.06
Unary	8	918.58	2.71	2.15	1.55
RabbitMQ Streams	4	504.41	769.51	3.21	6.88
RabbitMQ Streams	8	471.69	627.62	4.90	7.85
NATS JetStream	4	919.23	22.74	2.13	0.89
NATS JetStream	8	919.26	70.55	2.15	0.90
Kafka	4	909.64	377.43	0.01	8.16
Kafka	8	901.43	1,784.66	0.02	8.92

and its p_{95} inter-arrival delay stays near 1.55–1.59 ms. Unary keeps the lower end-to-end latency, with p_{95} values between 1.66 and 2.71 ms instead of 6.85–8.82 ms for client-streaming, but its pacing is less regular. NATS JetStream reaches a third operating point: it stays near 919 msg/s at both concurrency levels with sub-millisecond jitter, but its p_{95} latency rises to 22.74 ms at concurrency 4 and 70.55 ms at concurrency 8. Kafka lands close to the same throughput region as NATS, yet with a much less stable delivery rhythm: its p_{95} latency rises to 377.43 ms at concurrency 4 and 1,784.66 ms at concurrency 8, while jitter climbs above 8 ms in both cases. RabbitMQ Streams falls into a different regime altogether. Throughput drops to 472–504 msg/s and p_{95} latency rises to 628–770 ms, while inter-arrival jitter grows sharply. In other words, NATS keeps a brokered path relatively close to the target rate with moderate latency cost, Kafka keeps the target rate but pays for it with much heavier latency and pacing variance, and RabbitMQ no longer does so at a rate or latency level that matches the direct RPC paths.

b) Slow-consumer backpressure scenario.: The second focused experiment was a backpressure scenario. It used the same 1 KiB payload size with concurrency 8, but the single sink instance was slowed down by adding a 2 ms processing delay per message while the producer still targeted 4,000 messages per second. Because the sink can process at most $\lfloor 1000/2 \rfloor = 500$ messages per second per connection, the aggregate theoretical sink capacity is $500 \times 8 = 4{,}000$ messages per second, which means the scenario operates at the boundary of the sink's processing budget. Any transport overhead or scheduling jitter pushes the effective throughput below the requested rate, causing messages to accumulate either in the gRPC transport buffers of the transformer or in the RabbitMQ broker path. This scenario was designed to expose whether the transport absorbs overload through explicit throttling or through queue growth and accumulated delay.

This scenario shows that higher throughput is not automatically the better outcome. Under backpressure, client-streaming kept the highest realized throughput at 3,459 msg/s, but it did so by allowing the pipeline to accumulate a large queue: median end-to-end latency rose to about 1.31 seconds and p_{95}

Table VII
BACKPRESSURE SCENARIO WITH AN INTENTIONALLY SLOW SINK IN THE COMPLETED SINGLE-REPEAT SUITE.

Transport mode	Throughput	Median lat.	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	3,459.04	1,311.43	2,044.41	2.13	0.64
Unary	2,363.38	0.45	0.83	3.16	1.10
RabbitMQ Streams	478.94	198.11	413.74	3.06	8.81
NATS JetStream	3,092.38	1,551.54	3,807.50	2.14	0.66
Kafka	1,249.06	5,239.01	9,118.73	2.35	1.11

Table VIII
RECOVERY SCENARIO WITH A FORCED TRANSFORMER RESTART IN THE COMPLETED SINGLE-REPEAT SUITE.

Transport mode	Throughput	p_{95} lat.	Duplicates	Ordering	First post-fail.	E2E recovery	Backlog drain	Reached target
Client-streaming	1,467.13	28.38	9,145	0	0.00	6,000.00	10,632.03	Yes
Unary	597.21	1.66	0	0	23,819.02	24,000.00	30,489.03	Yes
RabbitMQ Streams	1,539.17	5,111.95	4,056	0	1,009.89	6,000.00	9,993.99	Yes
NATS JetStream	608.89	800.03	0	6	2.11	0.00	29,846.80	Yes
Kafka	589.41	28,234.42	0	0	1,667.00	30,932.12	30,932.12	No

latency to about 2.04 seconds. Unary behaved almost oppositely. It gave up throughput, but kept latency near the submillisecond range for successfully delivered requests. NATS JetStream moved close to the client-streaming regime rather than the RabbitMQ one: it sustained 3,092 msg/s, but paid for that with a 1.55-second median latency and a 3.81-second p_{95} latency. Kafka entered a harsher brokered regime under the same load. It sustained only 1,249 msg/s, while median latency rose above 5.2 seconds and p_{95} latency exceeded 9.1 seconds. RabbitMQ Streams throttled even harder than unary, sustaining only 479 msg/s. That stronger throttling kept its median and p_{95} latencies below the client-streaming row in this particular run, but the transport also exhibited a 21.2-second maximum latency and far higher jitter. The current interpretation is therefore that client-streaming and NATS preserve bulk movement by buffering aggressively, unary protects responsiveness by throttling, Kafka preserves correctness but sheds throughput only after a large backlog has already accumulated, and RabbitMQ Streams pays for brokered decoupling with a severe throughput collapse and a heavier long-tail backlog.

c) Forced-restart recovery scenario.: The third focused experiment was a recovery scenario. This run used 4 KiB messages, concurrency 8, a continuous target rate of 2,000 messages per second, and a forced *Transformer* restart after 3 seconds. The producer was allowed to retry up to 120 times with a 250 ms backoff. The purpose of this scenario was not to replace the main matrix, but to observe how the currently implemented transport mappings behave when the pipeline is interrupted during an active stream.

The recovery scenario now needs to be read from the sink rather than from the producer alone. Client-streaming resumed almost immediately after the forced restart: the first post-failure message arrived after roughly 0.003 ms, and the sink observed sustained recovery back to the target rate after 6 s. That continuity is not free, however. The current reconnect strategy replays each worker's sequence range, which produced 9,145 duplicate messages and a backlog-drain time of 10.6 s in this run. Unary shows the opposite behaviour. It remained duplicate-free and kept overall p_{95} latency low for delivered requests, but it did so while the pipeline stalled for much longer: the sink saw a 23.8 s delivery gap, 24.0 s to regain the target rate, and 30.5 s to fully drain the backlog.

RabbitMQ Streams changes the semantics rather than mirroring either gRPC row directly. Because the producer publishes into a broker, the forced *Transformer* restart is not observed as a producer-side reconnect event, so the producer-side recovery counters remain near zero and are useful only as diagnostics. The sink-side outcome is still not free. In the full rerun, RabbitMQ resumed much earlier than unary, with the first post-failure message arriving after 1.01 s, and it matched client-streaming on the 6 s sustained-recovery threshold while achieving the highest overall throughput in this single run. That better bulk continuity still comes with cost: the brokered path recorded a p_{95} latency of 5.11 s, a backlog-drain time of 9.99 s, and 4,056 duplicate messages. NATS JetStream reaches a different recovery trade-off again. In the refreshed rerun, the first post-failure message reached the sink after 2.11 ms and no duplicate deliveries were recorded, but throughput fell to 608.89 msg/s, backlog drain stretched to 29.85 s, and the sink still observed six ordering violations. Kafka preserves the same zero-duplicate and zero-ordering-violation outcome in this scenario, but with the slowest effective recovery profile in the suite: the first post-failure message arrived after 1.67 s, p_{95} latency rose to 28.23 s, backlog drain reached 30.93 s, and the sink never re-established the sustained-throughput threshold before the run ended. The brokered paths therefore differ not only in speed but in which semantic cost they pay during recovery.

5) Discussion: The current results support a more precise conclusion than a simple "streaming beats unary" claim. In the clean bulk-transfer baseline, client-streaming is the strongest choice for high-volume inter-service data movement, with the throughput advantage over unary varying from roughly $29.2\times$ for 256 B messages to $3.8\times$ for 16 KiB messages. Unary remains the lower-latency direct RPC option, with an average p_{95} latency of 3.01 ms across the final clean matrix, but it cannot match client-streaming's bulk transfer rate. This makes unary a useful control and a reasonable fit for small, latency-sensitive exchanges, not for large sustained data movement.

The brokered transports do not form one homogeneous category. NATS JetStream becomes the strongest brokered performer on average in the refreshed clean matrix once its mapping is tuned for explicit JetStream limits, async publish, batched pulls, and in-memory consumer state. It averages 13,116.27 msg/s and 32.13 MiB/s, outperforming both RabbitMQ Streams and Kafka on average throughput, and its average p_{95} latency falls to 1,581.11 ms. That result now comes with a different methodological caveat from the earlier draft: the old 1 KiB/concurrency-16 cliff no longer reproduces, but the clean rerun now exposes 5,086 duplicate deliveries concentrated in the 4 KiB concurrency-16 and 16 KiB high-concurrency rows. RabbitMQ Streams remains

a strong brokered reference, averaging 9,110.73 msg/s and 16.29 MiB/s, but it no longer leads the brokered pack in this rerun. Kafka now occupies a different position from the earlier draft: after enforcing key-hash partition affinity, periodic publisher flushes, and buffered consumer commits, it averages 7,354.42 msg/s and 20.12 MiB/s, overtakes unary overall, and remains architecturally attractive as the durable-log and replay-oriented option. Kafka therefore still carries a semantic caveat: no ordering violations remain, yet the 16 KiB concurrency-4 and concurrency-16 cells record nine duplicate deliveries in total. The main Kafka limitation in this single-node prototype is now tail behaviour and delivery stability in the largest-payload cases rather than outright throughput collapse.

The NATS debugging step is itself a useful methodological finding. The original NATS deployment allowed JetStream to infer host-scale memory while Kubernetes limited the container to 512 MiB, which produced OOM kills in the 16 KiB high-concurrency cells. Making JetStream memory and file-store limits explicit, reducing the per-stream storage cap, and allocating a 2 GiB pod memory limit made the full clean matrix complete without restarts. A second bottleneck remained in the benchmark mapping itself: switching NATS to async publish, batched pull consumption, and in-memory consumer state while keeping the stream file-backed changed the transport from a low-throughput outlier into the fastest brokered average performer. Even after that change, however, the broker pod itself stays lightly utilized in the clean matrix, peaking at only 2.2% CPU and 13 MiB, while the 1 KiB concurrency-16 row still exhibits severe queueing. That combination indicates the remaining instability is not explained by JetStream container exhaustion alone. This supports the transparency requirement of the benchmark: broker configuration is not incidental metadata, because it directly determines whether a result measures transport behaviour or accidental resource exhaustion.

The diagnostic stress scenarios reinforce why the final RQ1 answer should not be based on clean throughput alone. In the current five-transport scenario comparison, client-streaming preserved throughput under slow-consumer pressure by accumulating queueing delay, unary protected latency by sacrificing throughput, NATS behaved more like a high-throughput buffered path than a throttling one, RabbitMQ changed where backlog and recovery costs appeared, and Kafka showed the clearest durable-log trade-off: it kept duplicates and ordering violations at zero in the rerun scenarios, but paid for that with much heavier steady-rate latency, deeper backpressure queues, and the slowest recovery profile. These scenario rows justify the selected metrics: throughput tells the bulk-transfer story, latency percentiles expose backlog and tail effects, inter-arrival jitter captures pacing regularity, and duplicate/order counters reveal semantic costs during recovery.

Taken together, the current RQ1 result is a trade-off map. Client-streaming gRPC is the strongest default candidate for direct DYNAMOS-style bulk transfer when temporal coupling is acceptable. Unary gRPC remains the latency-oriented baseline and should not be replaced blindly for small request-like exchanges. RabbitMQ Streams remains a strong brokered reference when decoupling is needed without aggressive transport tuning. NATS JetStream is now the strongest brokered throughput candidate in the current prototype, but the refreshed reruns show that it should be treated as an explicitly at-least-once mapping with duplicate and occasional recovery-ordering costs rather than as a duplicate-free brokered default. Kafka is now a credible durable-log reference point rather than an obvious throughput outlier, and the completed scenario reruns clarify its actual niche in this prototype: it preserves recovery correctness better than RabbitMQ and NATS, but it does so with the heaviest latency and broker-resource cost among the brokered options.

G. Threats to Validity

Several threats to validity should be acknowledged when interpreting the current results.

a) Internal validity.: The current benchmark corpus is a single-repeat suite, so no confidence intervals or variance estimates are reported for the main tables. The results may therefore be affected by transient system-level effects such as CPU scheduling variance, garbage collection pauses, or short-lived background contention in the cluster. Short run durations also mean that the study may not capture warm-up effects or long-term stability behaviour that would emerge in sustained production workloads. The current NATS JetStream clean results already show this sensitivity: after the configuration and consumer-state fixes removed the earlier OOM failures, the clean rerun no longer reproduces the original 1 KiB/concurrency-16 cliff, but it now exposes duplicate-heavy heavier-payload rows instead. The current five-transport results now cover both the clean synthetic matrix and the focused scenario suite, but they still do so as single-pass observations rather than repeated measurements, so scenario-level conclusions should still be treated as provisional point estimates.

b) External validity.: The benchmark uses a synthetic three-stage pipeline with lightweight deterministic logic, which does not replicate the full complexity of a DYNAMOS deployment. Real DYNAMOS chains may involve additional processing stages, policy-enforcement sidecars, and heterogeneous service implementations. Furthermore, the tests are conducted in a single-node Kubernetes cluster, whereas production DYNAMOS deployments span multiple nodes with network partitioning and scheduling constraints that may affect latency and throughput characteristics differently. The Kafka and broker configurations are intentionally minimal single-node configurations, so their results should be read as prototype transport mappings rather than optimized production deployments. The Go-based benchmark services share the same runtime and gRPC library version, so results may not generalize to polyglot microservice chains.

c) Construct validity.: Jitter is operationalized as the standard deviation of inter-arrival times, which captures variability in delivery rhythm but does not distinguish between

periodic jitter and isolated outliers. Alternative metrics such as mean absolute deviation or percentile-based spread could complement the current definition. The clean matrix treats broker resource cost as measured Kubernetes CPU and memory usage, but it does not yet include broker-internal metrics such as Kafka log flush behaviour, JetStream store pressure, RabbitMQ stream segment state, or network traffic. Recovery is operationalized with sink-side time-to-first-message, end-to-end recovery to a sustained throughput threshold, backlog-drain time, and duplicate-related counters in the diagnostic stress presets. That definition still depends on the chosen 90% threshold, the three-window smoothing rule, and the estimated failure timestamp derived from the configured restart point rather than a transport-native fault signal.

IV. Planned: Scenario-Level Cross-Technology Comparison

The benchmark design and the completed five-transport clean matrix presented in section III address the first empirical part of *RQ1*. The benchmark harness now implements Apache Kafka and NATS JetStream under the same workload definitions, fairness constraints, and evaluation dimensions, and the focused CSV replay, continuous, backpressure, and recovery reruns are now available across the same five transports. The remaining empirical step for RQ1 is therefore to repeat the most important presets often enough to obtain variance estimates and to confirm that the current scenario trade-offs remain stable across runs. These results will allow the clean-matrix trade-off map to be extended from bulk-transfer performance to pacing, overload, recovery, and DYNAMOS-shaped payload realism with greater statistical confidence.

V. Planned: Architectural Integration Design

This section will address *RQ2*: how streaming communication can be integrated into DYNAMOS' microservice architecture while preserving dynamic service composition, policy-driven orchestration, and isolation guarantees. The work will examine how the selected streaming model aligns with the data-sharing archetypes implemented by DYNAMOS, specify how streaming channels are created and torn down within dynamically composed execution chains, define how sidecar responsibilities are divided for streaming versus non-streaming traffic, and address how long-lived connections interact with policy enforcement boundaries. The output will be a documented integration design accompanied by a prototype implementation that demonstrates feasibility.

VI. Planned: System-Level Evaluation

This section will address *RQ3*: what impact streaming communication has on scalability, performance, and robustness compared to the current request-response approach under realistic workloads in DYNAMOS. The benchmark logic and evaluation dimensions established in the current study will be reused, but applied within the full DYNAMOS context rather than in an isolated benchmark pipeline. The evaluation will assess whether the trade-offs identified in the current five-transport clean matrix and later scenario-level comparison carry over to the integrated system, or whether integration-specific effects such as policy enforcement overhead, sidecar latency, or orchestration coordination dominate.

VII. Conclusion

This thesis establishes the first empirical basis for evaluating streaming communication in DYNAMOS. The literature review narrows the relevant design space to direct gRPC streaming and broker-based alternatives such as Kafka, RabbitMQ Streams, and NATS JetStream. The completed single-repeat clean matrix shows that client-streaming gRPC is markedly stronger for sustained bulk transfer, while unary gRPC remains preferable when low per-message latency dominates. Among the brokered implementations, NATS JetStream currently gives the strongest average throughput in the prototype once JetStream limits and consumer-state handling are configured explicitly, but the refreshed clean rerun also shows that this higher-throughput mapping now exposes duplicate deliveries in the heavier rows instead of the earlier isolated queueing cliff. RabbitMQ Streams remains a strong brokered reference, and Kafka's durable-log model becomes competitive once partitioning, batching, and buffered consumer commits are configured correctly. The completed scenario reruns, however, also show the corresponding Kafka cost profile more clearly: duplicate-free recovery comes with much heavier tail latency, slower backlog drain, and higher broker memory use than the other brokered mappings. Rather than yielding a single winner, the present results identify a concrete trade-off space around throughput, latency, broker configuration, delivery semantics, and resource cost.

That evidence provides a defensible benchmark baseline for the remainder of the thesis, even though the current corpus is still a single-pass suite rather than a repeated statistical study. The planned next phases repeat the most decision-relevant presets for variance estimation, examine how the most promising options align with DYNAMOS' archetypes and policy model, and translate those findings into an integration design and system-level validation.

Appendix A
Benchmark Preset Definitions

This appendix documents the complete preset definitions used in the benchmark. Each preset specifies the workload family, payload configuration, concurrency level, target rate (where applicable), failure injection parameters, and resource overlay tier. The five presets are: *synthetic-clean* (bulk transfer baseline), *csv-replay-check* (realism validation), *synthetic-continuous* (steady-rate pacing), *synthetic-backpressure* (slow-consumer overload), and *synthetic-recovery* (forced-restart resilience).

Appendix B
Full Run Matrix

This appendix summarizes the canonical run scopes used in the current thesis draft. The body of the thesis presents the

Table IX
BENCHMARK PRESETS USED IN THE CURRENT TRANSPORT STUDY.

Preset	Key parameters	Purpose
`synthetic-clean`	Synthetic payloads from 256 B to 16 KiB, concurrency 1–16, no injected failure	Baseline bulk-transfer throughput, latency, and correctness matrix
`csv-replay-check`	Copied DYNAMOS CSV rows batched as 25 or 100 rows per message	Realism check against non-synthetic payload structure
`synthetic-continuous`	1 KiB payloads, 1,000 msg/s target, concurrency 4 and 8, one completed comparable run	Steady-rate pacing and jitter comparison
`synthetic-backpressure`	1 KiB payloads, 4,000 msg/s target, concurrency 8, 2 ms sink delay, one completed comparable run	Slow-consumer overload and queueing behaviour
`synthetic-recovery`	4 KiB payloads, 2,000 msg/s target, concurrency 8, transformer restart after 3 s, 120 retries at 250 ms backoff, one completed comparable run	Restart recovery, replay, and continuity analysis

most decision-relevant subsets, while the benchmark result directory stores the corresponding per-run JSON, NDJSON, and CSV exports generated by the execution scripts. The current chapter is based on the completed single-repeat five-transport `synthetic-clean` matrix, supplemented by single-repeat diagnostic scenario reruns across the same five transports.

Table X
EXECUTION SCOPE OF THE CURRENT BENCHMARK CORPUS.

Scope	Transports	Variable dimensions	Repetitions	Role in thesis
Synthetic clean matrix	Client-streaming, unary, rabbitmq-streams, nats-jetstream, kafka	Payload size and concurrency sweep	1	Main five-transport bulk-transfer baseline in Section III
CSV replay check	Client-streaming, unary, rabbitmq-streams, nats-jetstream, kafka	Batch size and concurrency subset	1	Realism validation available across all five transports
Focused continuous	Client-streaming, unary, rabbitmq-streams, nats-jetstream, kafka	Concurrency 4 and 8 at fixed 1 KiB and 1,000 msg/s	1	Diagnostic pacing and jitter evidence across all five transports
Focused backpressure	Client-streaming, unary, rabbitmq-streams, nats-jetstream, kafka	Fixed 1 KiB, concurrency 8, 4,000 msg/s, 2 ms sink delay	1	Diagnostic overload and queueing evidence across all five transports
Focused recovery	Client-streaming, unary, rabbitmq-streams, nats-jetstream, kafka	Fixed 4 KiB, concurrency 8, forced transformer restart	1	Diagnostic recovery and replay evidence across all five transports

APPENDIX C
DETAILED CLEAN-MATRIX RESULTS

This appendix records the full completed five-transport `synthetic-clean` matrix at the per-case level. The main chapter uses averages by transport and message size to keep the argument readable, but the tables below expose the exact throughput and latency values for every payload-size and concurrency combination in the current single-repeat suite.

APPENDIX D
RESOURCE UTILISATION DETAILS

This appendix summarizes resource metrics for the diagnostic Kubernetes scenario suite where CPU and memory data are most informative. For brokered modes, the broker is reported as a separate role because its buffering layer is a core part of the transport cost. The backpressure preset produced too few stable `kubectl top` samples to support a meaningful per-role comparison across all transports, so the appendix focuses on the currently available continuous and recovery scenarios across all five transports. The five-transport clean-matrix broker resource summary is reported in Table V.

Table XI
DETAILED SYNTHETIC-CLEAN MESSAGE THROUGHPUT (MSG/S) BY PAYLOAD SIZE AND CONCURRENCY.

Msg size	Concur.	Stream	Unary	RabbitMQ	NATS	Kafka
256 B	1	76,468	637	3,058	19,548	15,768
	4	118,209	1,899	25,812	24,292	10,876
	8	66,768	3,530	21,838	5,757	12,565
	16	40,970	4,279	1,553	28,496	10,660
1 KiB	1	28,171	696	17,810	16,976	10,760
	4	36,448	2,161	23,577	15,253	9,688
	8	28,743	3,163	1,045	6,350	5,103
	16	17,368	4,375	15,091	3,601	4,740
4 KiB	1	16,424	673	5,975	11,101	6,377
	4	19,838	1,834	8,928	11,211	6,903
	8	15,930	2,945	8,859	12,002	7,865
	16	9,239	3,823	7,860	5,360	4,965
16 KiB	1	2,872	635	2,233	4,497	2,737
	4	9,487	1,570	769	3,866	2,866
	8	9,138	1,698	466	6,507	2,797
	16	4,083	2,745	899	5,257	3,000

Table XII
DETAILED SYNTHETIC-CLEAN DATA THROUGHPUT (MIB/S) BY PAYLOAD SIZE AND CONCURRENCY.

Msg size	Concur.	Stream	Unary	RabbitMQ	NATS	Kafka
256 B	1	18.67	0.16	0.75	4.77	3.85
	4	28.86	0.46	6.30	5.93	2.66
	8	16.30	0.86	5.33	1.41	3.07
	16	10.00	1.04	0.38	6.96	2.60
1 KiB	1	27.51	0.68	17.39	16.58	10.51
	4	35.59	2.11	23.02	14.90	9.46
	8	28.07	3.09	1.02	6.20	4.98
	16	16.96	4.27	14.74	3.52	4.63
4 KiB	1	64.16	2.63	23.34	43.36	24.91
	4	77.49	7.16	34.88	43.79	26.97
	8	62.23	11.50	34.61	46.88	30.72
	16	36.09	14.93	30.70	20.94	19.39
16 KiB	1	44.88	9.92	34.89	70.27	42.76
	4	148.24	24.54	12.01	60.41	44.78
	8	142.78	26.53	7.28	101.67	43.71
	16	63.79	42.89	14.05	82.14	46.88

REFERENCES

[1] J. Stutterheim, "Dynamos: Dynamically adaptive microservice-based os: A middleware for data exchange systems," Master's thesis, University of Amsterdam, 2023.

[2] J. Stutterheim, A. Mifsud, and A. Oprescu, "Dynamos: Dynamic microservice composition for data-exchange systems, lessons learned,"

Table XIII
DETAILED SYNTHETIC-CLEAN p_{95} LATENCY (MS) BY PAYLOAD SIZE AND CONCURRENCY.

Msg size	Concur.	Stream	Unary	RabbitMQ	NATS	Kafka
256 B	1	44.4	1.9	1,699.4	602.1	205.8
	4	63.7	3.2	481.8	457.1	1,112.3
	8	97.6	2.2	590.8	3,296.7	999.7
	16	227.1	3.9	1,111.3	422.6	941.1
1 KiB	1	31.6	1.8	314.2	661.1	476.4
	4	183.9	1.6	496.3	955.7	797.5
	8	185.0	2.9	1,833.1	2,773.8	3,511.2
	16	681.1	4.5	786.6	5,249.4	3,607.1
4 KiB	1	23.3	1.7	1,526.0	979.1	768.8
	4	66.0	3.1	998.3	1,160.2	1,438.7
	8	333.7	2.4	1,173.3	1,023.2	1,304.8
	16	1,067.7	4.1	1,589.8	3,256.2	3,039.3
16 KiB	1	35.0	1.5	1,001.1	2,226.7	138.5
	4	153.8	3.3	13,731.6	3,890.3	3,515.6
	8	300.5	4.7	407.2	1,933.2	2,789.9
	16	390.7	5.5	6,236.1	2,448.1	3,022.5

Table XIV
DETAILED SYNTHETIC-CLEAN p_{99} LATENCY (MS) BY PAYLOAD SIZE AND CONCURRENCY.

Msg size	Concur.	Stream	Unary	RabbitMQ	NATS	Kafka
256 B	1	77.7	12.2	1,700.3	616.4	311.0
	4	65.8	15.5	507.5	501.6	1,210.1
	8	101.0	27.3	622.2	3,320.0	1,115.3
	16	266.6	48.5	1,123.1	487.0	1,097.6
1 KiB	1	57.0	13.1	338.3	723.4	484.5
	4	190.2	4.8	534.5	965.1	814.8
	8	249.8	41.2	1,909.0	2,865.2	3,624.5
	16	745.8	36.1	812.4	5,302.7	4,009.9
4 KiB	1	39.0	14.7	1,573.9	1,015.6	791.9
	4	76.3	23.9	1,191.1	1,182.2	1,516.1
	8	399.6	53.7	1,220.7	1,103.0	1,591.3
	16	1,379.0	64.8	1,934.5	3,348.0	3,167.0
16 KiB	1	44.8	15.4	1,041.4	2,361.8	294.2
	4	188.2	28.7	14,750.1	3,952.5	3,800.5
	8	347.6	33.9	598.8	2,012.5	3,173.7
	16	2,837.1	65.9	6,991.1	2,782.5	3,420.5

Table XV
CONTINUOUS SCENARIO CPU USAGE BY ROLE (AVG./PEAK %).

Mode	Concur.	Producer	Transformer	Sink	Broker
Client-streaming	4	23.40 / 23.40	8.99 / 24.20	5.61 / 14.90	–
Client-streaming	8	–	9.31 / 25.60	5.64 / 15.50	–
Unary	4	35.70 / 35.70	17.88 / 39.10	11.12 / 24.10	–
Unary	8	–	15.54 / 33.70	9.72 / 20.90	–
RabbitMQ Streams	4	7.92 / 8.30	9.87 / 15.60	6.91 / 10.90	29.88 / 43.30
RabbitMQ Streams	8	7.88 / 8.20	10.46 / 15.80	7.50 / 11.70	33.00 / 46.50
NATS JetStream	4	–	12.62 / 24.90	5.60 / 15.40	18.37 / 40.90
NATS JetStream	8	–	16.23 / 23.70	6.45 / 14.20	22.74 / 38.50
Kafka	4	–	4.68 / 5.70	0.69 / 1.90	51.52 / 52.90
Kafka	8	–	8.58 / 8.80	1.00 / 2.20	41.13 / 58.80

Table XVI
RECOVERY SCENARIO CPU USAGE BY ROLE (AVG./PEAK %).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	31.60 / 31.60	12.64 / 31.60	3.89 / 17.50	–
Unary	4.60 / 6.00	2.45 / 9.10	2.56 / 6.10	–
RabbitMQ Streams	8.84 / 10.20	13.38 / 17.10	7.51 / 13.90	31.76 / 49.90
NATS JetStream	0.30 / 0.30	17.29 / 25.20	7.28 / 16.20	20.99 / 43.50
Kafka	0.40 / 0.40	7.63 / 8.10	1.44 / 2.50	46.77 / 75.40

in *Proceedings of the IEEE International Conference on Software Architecture (ICSA)*, 2023.

[3] C. Starck and J. Ghofrani, "Improving load balancing of long-lived streaming rpcs for grpc-enabled inter-service communication," in *Proceedings of the 15th Central European Workshop on Services and their Composition (ZEUS 2023)*, ser. CEUR Workshop Proceedings, S. Böhm and D. Lübke, Eds., vol. 3386, Hannover, Germany, 2023, pp. 45–51. [Online]. Available: https://ceur-ws.org/Vol-3386/

[4] A. Sodankoor, A. Shenoy, B. M. Moger, M. Gowda, P. K. Auradkar, and S. Kalambur, "Benchmarking grpc protocol on various virtualization technologies," in *ICT4SD 2025*, ser. Lecture Notes in Networks and Systems. Springer Nature Switzerland AG, 2026, vol. 1654, pp. 316–329.

[5] A. G. Ibrahim, R. P. Lopes, J. Rufino, and P. Leitão, "On the impact of message brokers implementations in the choreography of microservices," in *Open Lecture Series 2A (OL2A 2025)*, ser. Communications in Computer and Information Science (CCIS). Springer Nature Switzerland AG, 2026.

[6] J. Nuikka, "Comparison of cloud native messaging technologies," Master's thesis, Tampere University, Faculty of Information Technology and Communication Sciences, May 2021. [Online]. Available: https://urn.fi/URN:NBN:fi:tuni-202104223314

[7] G. Coviello, K. Rao, C. G. D. Vita, G. Mellone, and S. Chakradhar, "Dataxc: Flexible and efficient communication in microservices-based stream analytics pipelines," in *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing (DASC) / PiCom / CBDCom / CyberSciTech*, 2022.

[8] J. Figueira and C. Coutinho, "Developing self-adaptive microservices," *Procedia Computer Science*, vol. 232, pp. 264–273, 2024.

[9] S. Neumaier, G. Havur, and T. Pellegrini, "Towards an architecture for policy-aware decentral dataset exchange," in *Proceedings of the Fifteenth International Conference on Advances in Semantic Processing (SEMAPRO 2021)*. Barcelona, Spain: IARIA, Oct. 2021. [Online]. Available: https://www.researchgate.net/publication/358816711_Towards_an_Architecture_for_Policy-Aware_Decentral_Dataset_Exchange

[10] G. Thiyagarajan, V. Bist, and P. Nayak, "Strengthening grpc security in microservices: A proxy-based approach for mtls, jwt, and rbac enforcement," *International Journal of Computer Applications*, vol. 187, no. 28, Aug. 2025.

[11] J. A. Rasheedh and S. Saradha, "Reactive microservices architecture using a framework of fault tolerance mechanisms," in *Proceedings of the Second International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2021.

[12] B. Guntupalli and S. Vamshi Ch, "Designing microservices that handle high-volume data loads," *International Journal of AI, Big Data, Computational and Management Studies*, vol. 4, no. 4, pp. 76–87, 2023.

[13] Y. Gan and C. Delimitrou, "The architectural implications of cloud microservices," *IEEE Computer Architecture Letters*, vol. 17, no. 2, pp. 155–158, 2018.

[14] Ritu, S. Arora, A. Bhardwaj, A. Kukkar, and S. Kaur, "A comparative analysis of communication efficiency: Rest vs. grpc in microservice-based ecosystems," in *Proceedings of the 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*. IEEE, 2024.

[15] V. Lähtevänoja, "Communication methods and protocols between microservices on a public cloud platform," Master's thesis, Aalto University, School of Science, Espoo, Finland, May 2021. [Online]. Available: https://urn.fi/URN:NBN:fi:aalto-202106207501

[16] S. Ris, J. Araujo, and D. Beserra, "A systemic mapping of methods and tools for performance analysis of data streaming with container-

Table XVII
RECOVERY SCENARIO MEMORY USAGE BY ROLE (AVG./PEAK MIB).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	7.00 / 7.00	3.40 / 7.00	2.33 / 7.00	–
Unary	7.00 / 7.00	3.08 / 7.00	4.53 / 7.00	–
RabbitMQ Streams	10.00 / 10.00	25.72 / 31.00	5.78 / 8.00	490.91 / 511.00
NATS JetStream	8.00 / 8.00	14.00 / 14.00	9.75 / 15.00	306.06 / 537.00
Kafka	10.00 / 10.00	8.00 / 8.00	6.53 / 9.00	449.71 / 478.00

ized microservices architecture," in *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*, 2023, pp. 1–6.

[17] gRPC Authors. (2024) Core concepts, architecture and lifecycle. Last modified 2024-11-12. [Online]. Available: https://grpc.io/docs/what-is-grpc/core-concepts/

[18] M. Roohitavaf, K. Ren, G. Zhang, and S. Ben-Romdhane, "Logplayer: Fault-tolerant exactly-once delivery using grpc asynchronous streaming," 2019, arXiv preprint.

[19] Apache Software Foundation. (2026) Apache kafka. [Online]. Available: https://kafka.apache.org/

[20] P. Dobbelaere and K. Sheykh Esmaili, "Kafka versus rabbitmq," 2017, arXiv preprint.

[21] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," 2025, arXiv preprint.

[22] B. Çiftçi and B. Çiloğlugil, "A review of comparative studies on performance evaluation of communication mechanisms for microservices," in *Computational Science and Its Applications – ICCSA 2025*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland AG, 2025, vol. 15650, pp. 98–113.

[23] T. Sharvari and K. Sowmya Nag, "A study on modern messaging systems: Kafka, rabbitmq and nats streaming," 2019, arXiv preprint.

[24] RabbitMQ Team. (2026) Streams and superstreams (partitioned streams). [Online]. Available: https://www.rabbitmq.com/docs/streams

[25] The NATS Authors. (2026) Overview. [Online]. Available: https://docs.nats.io/nats-concepts/overview

[26] ——. (2026) Request-reply. [Online]. Available: https://docs.nats.io/nats-concepts/core-nats/reqreply

[27] ——. (2026) Jetstream. [Online]. Available: https://docs.nats.io/nats-concepts/jetstream

[28] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proceedings of the IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1519–1530.

[29] S. Henning and W. Hasselbring, "How to measure scalability of distributed stream processing engines?" in *Companion of the ACM/SPEC International Conference on Performance Engineering (ICPE '21)*. ACM, 2021, pp. 85–88.